CORE103L Computational Thinking
LW4 - Iteration



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.2 Marking scheme 2
1.2.1 Lab Objectives 2
1.3 Late submission policy 2
1.4 Use of AI 2
1.5 Viva 3

2 Prerequisites **4**
2.1 Assert Statements 4
2.2 Pytest 4

3 Lab Exercises **6**
3.1 Campus Elevator Floor Check 6
3.2 Campus Event Snack Sharing 8
3.3 Collatz-Style Sequence 9
3.4 Campus Word Game 10
3.5 Locker Number Puzzle 11
3.6 Campus Event Display Board 12

Introduction

1.1 Instructions

- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW4_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 10.

1.2.1 Lab Objectives

The following are the lab objectives of this lab:

- Iteration in Python

1.3 Late submission policy

There is no late submission policy for the lab.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, 'x should be a non-negative number'
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

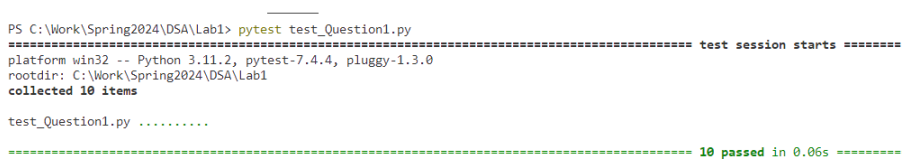
After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

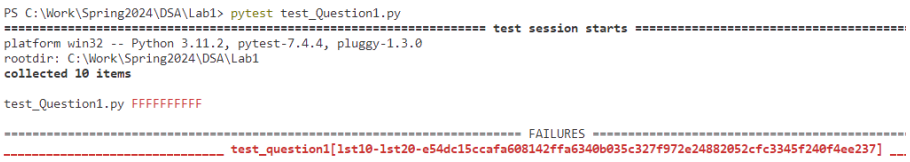
If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.



However, if one or more tests fail, you should receive a similar output.



Lab Exercises

3.1 Campus Elevator Floor Check

This question should be implemented in the file ***q1.py****.*

Before the campus building opens, the facilities team tests an elevator display. Floors above ground are represented by positive integers, the ground floor is `0`, and basement floors are represented by negative integers.

During the test, the display should show every floor number starting from starting_floor and moving down to ending_floor, including both endpoints.

Constraints

- $-5 \leq ending_floor < starting_floor \leq 20$

Function Specification

Write an **iterative** function named **countdown** that takes two parameters, **starting_floor** and **ending_floor**. If both arguments are integers, the function must print all floor numbers from starting_floor down to ending_floor, inclusive.

Print the numbers on **one line**, separated by a comma followed by one space. Do not print a trailing comma after the final number.

If either argument is not an integer, the function must print:

```
Error: bad argument. countdown is defined for integers only.
```

Type Checking

You may use Python's built-in `type()` function to check whether an argument is an integer.

```
>>> type(3) == int
True

>>> type(3.14) == int
False
```

Output Formatting

Python's `print()` function normally moves to a new line after printing. You may use the `end` parameter to control what is printed after a value.

```
print(1, end=', ')
print(2, end=', ')
print(3)
```

This produces:

```
1, 2, 3
```

Notice that the final value is printed differently so that the output does not end with an extra comma.

Sample Interaction

```
>>> countdown(5, 2)
5, 4, 3, 2

>>> countdown(4, -3)
4, 3, 2, 1, 0, -1, -2, -3

>>> countdown(-1, -5)
-1, -2, -3, -4, -5

>>> countdown(3.0, -4)
Error: bad argument. countdown is defined for integers only.

>>> countdown(3, -4.5)
Error: bad argument. countdown is defined for integers only.
```

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q1.py
```

3.2 Campus Event Snack Sharing

This question should be implemented in the file ***q2.py****.*

A student society is preparing refreshments for a long campus event. It has a fixed number of juice boxes and snack bars and wants to serve as many guests as possible. Every guest must receive the same positive number of juice boxes and the same positive number of snack bars, with nothing left over.

Given the numbers of juice_boxes and snack_bars, determine the maximum number of guests that can be served equally.

Constraints

- juice_boxes and snack_bars are positive integers.
- `1 <= juice_boxes <= 1000`
- `1 <= snack_bars <= 1000`
- Do not use any built-in function that directly computes the greatest common divisor.

Function Specification

Write an **iterative** function **max_equal_guests** that takes the integer parameters **juice_boxes** and **snack_bars** and returns the maximum number of guests among whom both item quantities can be divided evenly.

Sample Interaction

```
>>> max_equal_guests(12, 18)
6

>>> max_equal_guests(9, 9)
9
```

In the first case, 6 guests can each receive 2 juice boxes and 3 snack bars, so the function returns 6.

In the second case, 9 guests can each receive one juice box and one snack bar, so the function returns 9.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q2.py
```

3.3 Collatz-Style Sequence

This question should be implemented in the file `q3.py`.

The standard Collatz conjecture repeatedly applies this rule: if n is even, replace it with $n/2$; if n is odd, replace it with $3n + 1$. It asks whether every positive starting integer eventually reaches 1.

We will use a **different Collatz-style variant**: odd values use $3n - 1$, and the process stops when the current value becomes 1, 5, or 17.

Function Specification

Write an **iterative** function named **collatz_sequence** with one integer parameter, **starting_number**. The function must print the sequence generated by the following algorithm. Let the current number initially be starting_number:

1. Print the current number as part of the sequence.
2. If the current number is 1, 5, or 17, stop.
3. If the current number is even, replace it with `current_number // 2`. Otherwise, replace it with `3 * current_number - 1`.
4. Go back to Step 1.

The entire sequence must be printed on **one line**, with consecutive values separated by a comma followed by one space. Do not print a trailing comma after the final value.

Constraints

- starting_number is an integer.
- `starting_number > 0`
- You may assume that every test case provided will eventually reach 1, 5, or 17.

Sample Interaction

```
>>> collatz_sequence(1)
1

>>> collatz_sequence(3)
3, 8, 4, 2, 1
```

For `starting_number = 1`, the function prints 1 and stops immediately.

For `starting_number = 3`, the values are $3 \to 8 \to 4 \to 2 \to 1$, so the final output is `3, 8, 4, 2, 1`.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q3.py
```

3.4 Campus Word Game

This question should be implemented in the file ***q4.py****.*

A student society is playing a word game in its group chat. In the game, a lowercase word is called `Happy` if it contains **at least three consecutive vowels**. Otherwise, the word is called `Sad`.

Constraints

- word contains lowercase English letters only.
- word contains at least one character.

Function Specification

Write an **iterative** function **word_mood** that takes the parameter **word** and returns exactly the string `"Happy"` or `"Sad"`. The English vowels are `a`, `e`, `i`, `o`, and `u`.

Sample Interaction

```
>>> word_mood('abxy')
Sad

>>> word_mood('abcdeeafg')
Happy
```

In the first case, `word = abxy` does not contain three consecutive vowels, so the function returns `"Sad"`.

In the second case, `word = abcdeeafg` contains the three-consecutive-vowel substring `eea`, so the function returns `"Happy"`.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q4.py
```

3.5 Locker Number Puzzle

This question should be implemented in the file `q5.py`.

During orientation, students are given a quick puzzle based on numeric locker IDs: take a locker number and read its digits in reverse order.

Given a positive integer locker_number, reverse its digits and return the reversed integer.

Constraints

- locker_number is a positive integer.
- `1 <= locker_number <= 10**5`
- **Solve this question arithmetically using iteration.**
- **You are not allowed to convert locker_number to a string.**

Function Specification

Write an **iterative** function **reverse_number** that takes a positive integer **locker_number** as a parameter and returns the integer formed by reversing its digits.

Sample Interaction

```
>>> reverse_number(12345)
54321

>>> reverse_number(33602)
20633

>>> reverse_number(1200)
21
```

For `locker_number = 12345`, reversing the digits gives 54321.

For `locker_number = 33602`, reversing the digits gives 20633.

For `locker_number = 1200`, leading zeros in the reversed form are dropped. Hence, reversing the digits gives 21.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q5.py
```

3.6 Campus Event Display Board

This question should be implemented in the file ***q6.py****.*

A student society is preparing display boards for a campus event. Ali can carry a board only if its weight does not exceed his carrying limit. Among all the boards he can carry, Ali wants to choose the one with the largest display area.

Ali considers board_count available boards. For each board, its board_length, board_width, and board_weight are provided.

Constraints

- `1 <= board_count <= 4`
- `1 <= weight_limit <= 100`
- `1 <= board_weight <= 100`
- `1 <= board_length <= 100`
- `1 <= board_width <= 100`

Function Specifications

1. Write a helper function **area_calculator** that takes positive integers **board_length** and **board_width** as parameters and returns the area of the board.
2. Write a main function **best_choice** that takes integer **board_count**, representing the number of available boards, and integer **weight_limit**, representing the maximum weight Ali can carry. For each board, the function must determine whether Ali can carry it. If the board is within the weight limit, use **area_calculator** to calculate its area and keep track of the largest such area.

   After all boards have been processed, print the largest area among the boards Ali can carry.

   If Ali cannot carry any of the available boards, print `"No Board"`.

Interaction

board_count and weight_limit are supplied as arguments when the function **best_choice** is called. During the function call, the function reads exactly `3 * board_count` input values.

For each board, in order:

- one line contains board_length,
- the next line contains board_width,
- the next line contains board_weight.

The function prints the largest area among the boards Ali can carry. If none of the boards are within the weight limit, it prints `"No Board"`.

Sample Interaction

In the sample below, lines written as `[Input -> value]` represent values entered by the user while `best_choice` is running. They are **not** output produced by the function.

```
>>> best_choice(3, 6)
[Input -> 3]
[Input -> 4]
[Input -> 4]
[Input -> 5]
[Input -> 5]
[Input -> 7]
[Input -> 5]
[Input -> 2]
[Input -> 5]
12

>>> best_choice(2, 6)
[Input -> 3]
[Input -> 6]
[Input -> 8]
[Input -> 5]
[Input -> 4]
[Input -> 9]
No Board
```

In the first case, Ali can carry boards weighing at most 6 kg. The first board weighs 4 kg and has area $3 \times 4 = 12$, so Ali can carry it. The second board has area $5 \times 5 = 25$, but it weighs 7 kg, so Ali cannot carry it. The third board weighs 5 kg and has area $5 \times 2 = 10$. Therefore, the largest area among the boards Ali can carry is 12, so the function prints 12.

In the second case, Ali's weight limit is 6 kg, but both boards weigh more than 6 kg. Therefore, the function prints `"No Board"`.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q6.py
```